

Week 8 Practical

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
divya@Divyasree:~$ mkdir memory_management
divya@Divyasree:~$ cd memory_management
divya@Divyasree:~/memory_management$ ^C
divya@Divyasree:~/memory_management$ nano memory_management.c
divya@Divyasree:~/memory_management$ gcc -Wall -g memory_management.c -o memory_management
divya@Divyasree:~/memory_management$ ./memory_management
```

```
1. malloc() allocation
Values after malloc: 10 20 30 40 50

2. calloc() allocation
Values after calloc: 0 0 0 0 0

3. realloc() allocation
Values after realloc: 10 20 30 40 50 60 70 80 90 100
```

```
4. free() memory
Memory successfully freed.
divya@Divyasree:~/memory_management$ valgrind --version
valgrind-3.22.0
divya@Divyasree:~/memory_management$ sudo apt update
sudo apt install valgrind
[sudo] password for divya:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1066 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1337 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [301 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [220 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1699 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [341 kB]
```

```
Get:8 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [220 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1699 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [341 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1689 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [386 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:16 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5760 B]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.6 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1216 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [244 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]
Get:22 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1566 kB]
Get:23 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [361 kB]
Fetched 11.5 MB in 13s (863 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
97 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
valgrind is already the newest version (1:3.22.0-0ubuntu3).
0 upgraded, 0 newly installed, 0 to remove and 97 not upgraded.
divya@Divyasree:~/memory_management$ valgrind --version
valgrind-3.22.0
divya@Divyasree:~/memory_management$ valgrind --leak-check=full --show-leak-kinds=all ./memory_management
==1242== Memcheck, a memory error detector
==1242== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1242== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1242== Command: ./memory_management
```

```

divya@Divyasree:~/memory_management$ valgrind --version
valgrind-3.22.0
divya@Divyasree:~/memory_management$ valgrind --leak-check=full --show-leak-kinds=all ./memory_management
==1242== Memcheck, a memory error detector
==1242== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1242== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1242== Command: ./memory_management
==1242==
1. malloc() allocation
Values after malloc: 10 20 30 40 50

2. calloc() allocation
Values after calloc: 0 0 0 0 0

3. realloc() allocation
Values after realloc: 10 20 30 40 50 60 70 80 90 100

4. free() memory
Memory successfully freed.
==1242==
==1242== HEAP SUMMARY:
==1242==   in use at exit: 0 bytes in 0 blocks
==1242==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==1242==
==1242== All heap blocks were freed -- no leaks are possible
==1242==
==1242== For lists of detected and suppressed errors, rerun with: -s
==1242== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
divya@Divyasree:~/memory_management$

```

Code :

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // malloc()
    printf("1. malloc() allocation\n");
    int *arr = (int *)malloc(5 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++) {
        arr[i] = (i + 1) * 10;
    }

    printf("Values after malloc: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n\n");

    // calloc()
    printf("2. calloc() allocation\n");
    int *zeros = (int *)calloc(5, sizeof(int));
}

```

```

GNU nano 7.2 memory_management.c
    if (zeros == NULL) {
        printf("Memory allocation failed\n");
        free(arr);
        return 1;
    }

    printf("Values after calloc: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", zeros[i]);
    }
    printf("\n\n");

    // realloc()
    printf("3. realloc() allocation\n");
    arr = (int *)realloc(arr, 10 * sizeof(int));

    if (arr == NULL) {
        printf("Reallocation failed\n");
        free(zeros);
        return 1;
    }

    for (i = 5; i < 10; i++) {
        arr[i] = (i + 1) * 10;
    }

    printf("Values after realloc: ");
    for (i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n\n");

    // free()
    printf("4. free() memory\n");
    free(arr);
    free(zeros);

    printf("Memory successfully freed.\n");

    return 0;
}

```

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```

divya@Divyasree:~$ mkdir copy_on_write
divya@Divyasree:~$ cd copy_on_write
divya@Divyasree:~/copy_on_write$ nano cow_demo.c
divya@Divyasree:~/copy_on_write$ gcc -Wall -O0 cow_demo.c -o cow_demo
divya@Divyasree:~/copy_on_write$ ./cow_demo
Parent PID: 2195
10 MB memory allocated and initialized.

Child Process
Child PID: 2196
Before modification: data[0] = A
Child modifying memory...

Parent Process
Child PID: 2196
Parent data[0] = A
Parent waiting while child modifies memory...
After modification: data[0] = B
Child completed modification.
Parent data[0] is still: A
Child process completed.
divya@Divyasree:~/copy_on_write$ ./cow_demo
Parent PID: 2197
10 MB memory allocated and initialized.

Parent Process
Child PID: 2198
Parent data[0] = A
Parent waiting while child modifies memory...

Child Process
Child PID: 2198
Before modification: data[0] = A

```

```

After modification: data[0] = B
Child completed modification.
Parent data[0] is still: A
Child process completed.
divya@Divyasree:~/copy_on_write$ ps aux | grep cow_demo
divya      2201  0.0  0.0  4104  2100 pts/0    S+   15:54   0:00 grep --color=auto cow_demo
divya@Divyasree:~/copy_on_write$ grep -E "VmSize|VmRSS|RssAnon" /proc/1234/status
grep: /proc/1234/status: No such file or directory
divya@Divyasree:~/copy_on_write$ pmap 1234
divya@Divyasree:~/copy_on_write$ |

```

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (10 * 1024 * 1024) // 10 MB

int main() {
    char *data;

    printf("Parent PID: %d\n", getpid());

    // Allocate 10 MB
    data = (char *)malloc(SIZE);

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    // Initialize memory so that physical pages are allocated
    for (int i = 0; i < SIZE; i++) {
        data[i] = 'A';
    }

    printf("10 MB memory allocated and initialized.\n");

    pid_t pid = fork();

```

```

GNU nano 7.2 cow_demo.c
if (pid == 0) {
    // Child process
    printf("\nChild Process\n");
    printf("Child PID: %d\n", getpid());

    printf("Before modification: data[0] = %c\n", data[0]);

    // Modify memory in child
    printf("Child modifying memory...\n");

    for (int i = 0; i < SIZE; i++) {
        data[i] = 'B';
    }

    printf("After modification: data[0] = %c\n", data[0]);

    printf("Child completed modification.\n");

    sleep(5);

    free(data);
    exit(0);
} else {
    // Parent process
    printf("\nParent Process\n");
    printf("Child PID: %d\n", pid);

    printf("Parent data[0] = %c\n", data[0]);

    printf("Parent waiting while child modifies memory...\n");

    sleep(2);

    printf("Parent data[0] is still: %c\n", data[0]);

    wait(NULL);

    printf("Child process completed.\n");

    free(data);
}

return 0;
}

```